Listing 8.5 Kafka service

```
const { Kafka } = require('kafkajs');

const clientId = 'movie-event';

const kafka = new Kafka({
  clientId: clientId,
  brokers: ['localhost:9092']
});

const consumer = kafka.consumer({ groupId: clientId });

const consumeMovieStream = async (handler) => {
  await consumer.connect();
  await consumer.subscribe({ topic: 'movies', fromBeginning: false });
  await consumer.run({
    eachMessage: async ({ topic, partition, message }) => {
      try {
        await handler(JSON.parse(message.value.toString()));
      } catch (e) {
        console.error('unable to handle message', e);
      }
    },
  })
}

module.exports = consumeMovieStream;
```

The `consumeMovieStream` function exported in listing 8.5 takes an argument, which will be the movie handler function shown previously in listing 8.2. Call the consume movie function within the server initialization function, as follows.

Listing 8.6 Server

```
const app = require('express')();
const cors = require('cors');
const consumeMovieStream = require('./src/service/kafka');
const streamHandler = require('./src/movies/movie.handler');
const port = 8080;

const init = () => {
    app.use(cors());

    consumeMovieStream(streamHandler);

    return app.listen(port, () =>
      console.log(`Provider API listening on port ${port}...`));
};

init();
```

Figure 8.5 shows the whole event handler implementation together, from the point at which the event is produced to the insertion of the data to the database.